Autonomous Mobile Robot for 3D Mapping and Disaster Assessment in GPS Denied Environments

1st Somya Jain
Dept. of Computing Technologies,
SRM Institute of Science and Technology,
Chennai, India.
jainsomya3011@gmail.com

2nd Rounaq Moin
Dept. of Computing Technologies,
SRM Institute of Science and Technology,
Chennai, India.
rm3062@srmist.edu.in

3rd Dr. J. Briskilal
Dept. of Computing Technologies,
SRM Institute of Science and Technology,
Chennai, India.
briskilj@srmist.edu.in

Abstract*— Disaster-affected environments such as collapsed buildings and earthquake zones are highly unpredictable and dangerous for human rescuers. Limited visibility, unstable terrain, and the absence of GPS or communication infrastructure significantly reduce situational awareness during rescue operations. This paper presents an autonomous mobile robot system designed for real-time 3D mapping and disaster assessment in GPS-denied environments. The proposed system integrates a 2D LiDAR sensor with odometry-based localization using the Robot Operating System 2 (ROS2) framework to perform autonomous navigation and obstacle avoidance entirely onboard. A waypoint-based navigation algorithm guides the robot through constrained environments using dynamic obstacle detection and adaptive path planning. The robot autonomously explores unknown environments while avoiding obstacles using a sensor fusion approach combining LiDAR range data with wheel odometry. The system is evaluated through simulation experiments in Gazebo that replicate disaster-like conditions, including U-shaped obstacle courses representing collapsed structures. Results demonstrate accurate localization with average positioning error under 0.1m, reliable obstacle detection at distances up to 10 meters, and successful navigation through confined spaces with clearances as narrow as 1.5 meters. This work demonstrates the practical feasibility of deploying low-cost autonomous robots to enhance disaster response efficiency while reducing risks to human responders.

Keywords*— Autonomous robot, Mobile robot, SLAM, 2D mapping, Disaster assessment, GPS-denied navigation, ROS2, Waypoint navigation, Obstacle avoidance

I. INTRODUCTION

Natural disasters such as earthquakes, landslides, and structural collapses often create environments that are extremely hazardous and difficult to navigate. In the first critical hours after a disaster, rescue teams must operate with limited information about the internal structure of collapsed buildings, blocked pathways, and potential survivor locations. Manual exploration in such conditions is slow, risky, and can lead to secondary accidents. According to disaster response statistics, approximately 25% of rescue personnel injuries occur during reconnaissance operations in unstable structures [1].

Robotic systems have been introduced to assist disaster response, but many existing solutions face significant limitations. Aerial drones provide rapid visual coverage but cannot access confined or underground spaces and have limited flight endurance. Cloud-dependent robots fail when communication networks or GPS signals are unavailable, which is common during disasters. Teleoperated systems require constant human attention and suffer from communication delays, reducing operational efficiency.

To overcome these limitations, this paper proposes an autonomous mobile robot capable of operating independently in GPS-denied and communication-constrained environments. The system is built on the Robot Operating System 2 (ROS2) framework using the Pioneer P3-DX robot platform equipped with a Hokuyo 2D LiDAR sensor. By combining sensor data with onboard navigation algorithms, the robot can autonomously explore unknown environments and avoid obstacles without external infrastructure.

The key innovation of this work lies in its practical implementation of waypoint-based autonomous navigation using readily available hardware and open-source software. Unlike complex multi-sensor SLAM systems that require expensive computing resources, our approach demonstrates that effective disaster reconnaissance can be achieved using a single 2D LiDAR and basic odometry. The navigation algorithm employs a state machine architecture with five operational modes (Move, Avoid, Wall-Follow, Recover, Done) that enables robust behavior in cluttered environments.

The primary goal of this work is to create a practical, deployable robotic system that improves rescue efficiency while minimizing risk to human responders. The system is designed for rapid deployment scenarios where setup time is critical and infrastructure is unavailable.

Contributions of this work include: - Design and implementation of a fully autonomous single-robot disaster assessment system using ROS2 and Gazebo simulation environment. Development of a waypoint-based navigation algorithm with dynamic obstacle avoidance suitable for confined spaces. Integration of 2D LiDAR-based perception with differential drive control for real-time navigation. Experimental validation in simulated disaster-like environments including U-shaped obstacle courses that replicate collapsed structure geometry. Demonstration of practical deployment feasibility using low-cost hardware suitable for disaster response applications.

The remainder of this paper is organized as follows: Section II reviews related work in disaster robotics and autonomous navigation. Section III provides an overview of the system design. Section IV details the system architecture including hardware and software components. Section V describes the implementation methodology including the navigation algorithm and obstacle avoidance strategies. Section VI presents the experimental setup and evaluation metrics. Section VII discusses results from simulation experiments. Section VIII concludes with lessons learned and future research directions.

II. RELATED WORKS

Robotic assistance for disaster response has been actively researched since the 1990s, with significant deployments following major disasters including the World Trade Center collapse (2001), Fukushima nuclear disaster (2011), and various earthquake responses worldwide. Murphy [2] provides a comprehensive survey of rescue robotics, categorizing systems into aerial, ground, and hybrid platforms

Aerial drones have become popular for rapid reconnaissance due to their speed and aerial perspective. However, they face fundamental limitations in disaster scenarios: limited battery life (typically 15-30 minutes), inability to access enclosed spaces, and susceptibility to GPS denial in urban canyon environments. The DJI Mavic and similar commercial drones used in disaster response cannot penetrate building structures where survivors are most likely trapped [3].

Ground-based robots offer complementary capabilities. The PackBot platform [9], widely deployed in military and disaster scenarios, demonstrates the value of ruggedized tracked robots for terrain traversal. However, PackBot and similar systems are teleoperated rather than autonomous, requiring constant human control and reliable communication links. During the Fukushima disaster, communication failures limited robot effectiveness despite their radiation hardness [4].

Simultaneous Localization and Mapping (SLAM) has been extensively studied as the fundamental enabler of autonomous navigation. Thrun et al. [1] established the theoretical foundations of probabilistic robotics and SLAM algorithms. Modern SLAM approaches can be categorized into vision-based, LiDAR-based, and multi-sensor fusion methods.

Vision-based SLAM systems such as ORB-SLAM [2] and ORB-SLAM2 [3] achieve impressive accuracy using monocular or stereo cameras. These systems extract visual features and track them across frames to estimate camera motion. However, vision-based methods struggle in low-light conditions, texture less environments, and dusty atmospheres—all common in disaster scenarios. Mur-Artal and Tardós demonstrated that ORB-SLAM2 can fail completely in poorly lit environments [3].

LiDAR-based SLAM provides robust performance across varied lighting conditions. LOAM (LiDAR Odometry and Mapping) [4] achieves real-time 3D mapping using Velodyne sensors but requires significant computational resources. Cartographer, developed by Google, provides 2D and 3D SLAM for ROS-based robots [11]. While highly accurate, these systems require careful tuning and significant processing power.

Recent research emphasizes visual-inertial fusion to improve robustness. VINS-Mono [6] combines monocular camera data with IMU measurements to reduce drift and improve localization in challenging conditions. Qin et al. demonstrated localization accuracy under 0.3% of travel distance in indoor environments. However, sensor fusion systems increase complexity and cost, potentially reducing reliability in harsh disaster environments.

Autonomous navigation requires not only localization but also reactive obstacle avoidance. Classical approaches include artificial potential fields, dynamic window approaches, and sampling-based methods. The Vector Field Histogram (VFH) algorithm, widely used in mobile robotics, constructs a polar histogram of obstacles and selects safe heading directions [12].

More recent work employs learning-based approaches. Deep reinforcement learning has shown promise for end-to-end navigation from sensor inputs to motor commands [13]. However, learning-based methods require extensive training data and may not generalize well to novel disaster environments.

The Robot Operating System (ROS) [8] has become the de facto standard for robotics research and development. ROS2, the next generation, addresses real-time performance and security limitations of ROS1 [14]. Navigation Stack 2 (Nav2) provides production-ready autonomous navigation capabilities for ROS2 robots, including global and local path planning, cost map generation, and recovery behaviours [15].

Several disaster response robots have been built using ROS. The DARPA Robotics Challenge featured multiple ROS-based humanoid robots attempting disaster response tasks [16]. However, most systems still rely on human operators for high-level decision making.

Despite extensive research, significant gaps remain in practical disaster robotics: Complexity vs. Reliability: Many state-of-the-art systems are too complex for rapid deployment in chaotic disaster scenarios. Infrastructure Dependencies: Most systems assume availability of GPS, communications, or cloud computing. Cost Barriers: High-end sensor suites (Velodyne LiDAR, stereo cameras) limit widespread deployment. Sim-to-Real Gap: Many algorithms work well in simulation but fail in real-world disaster conditions.

Our work addresses these gaps by emphasizing practical deploy ability over maximum performance. Rather than using expensive 3D LiDAR or camera arrays, we demonstrate effective navigation using a single 2D LiDAR sensor. Instead of complex SLAM back-ends with loop closure, we employ straightforward odometry with periodic correction. The system is designed for rapid deployment with minimal calibration.

Several disaster response robots have been built using ROS. The DARPA Robotics Challenge featured multiple ROS-based humanoid robots attempting disaster response tasks [16]. However, most systems still rely on human operators for high-level decision making.

This approach trades absolute accuracy for robustness and simplicity. While research systems may achieve centimetre-level localization, disaster response often requires only meter-level accuracy to identify blocked passages, structural hazards, and potential survivor locations. Our focus on practical feasibility over theoretical optimality represents a complementary direction for disaster robotics research.

III. SYSTEM OVERVIEW OF AUTONOMOUS MOBILE ROBOT

The system design follows three core principles: simplicity, autonomy, and deploy ability. Simplicity focuses on minimizing complexity to maximize reliability in unpredictable environments. Autonomy ensures that the robot can operate independently without GPS, communications, or human intervention. Deploy ability emphasizes the use of affordable, readily available hardware that can be quickly deployed. These principles guided all design decisions, from sensor selection to algorithm implementation. Unlike research systems that prioritize maximum accuracy, this approach prioritizes operational reliability under real-world constraints.

The intended operational scenario is early-stage disaster reconnaissance. In the deployment phase, the robot is placed near the disaster site entrance. During goal setting, a human operator designates a reconnaissance waypoint inside the disaster zone. The robot then performs autonomous exploration by navigating to the waypoint while avoiding obstacles and mapping the environment. Upon completion or return, data retrieval is performed by downloading the robot's recorded map and sensor logs for analysis. This scenario assumes degraded infrastructure but requires only initial goal specification, allowing the robot to operate autonomously during the reconnaissance phase and reduce risk to human personnel.

The implemented system provides several key capabilities. It enables autonomous waypoint navigation, allowing the robot to reach predefined goal positions without human intervention. It performs real-time obstacle detection using a 2D LiDAR sensor with detection ranges up to 30 meters. The robot supports dynamic obstacle avoidance by reacting to obstacles through turning away, wall-following, or backing up and re-planning. It is capable of confined space navigation through passages as narrow as 1.5 meters using adaptive speed control. The system can recover from stuck situations by detecting immobility and executing recovery maneuvers such as reversing, rotating, and advancing. Continuous localization is maintained using wheel odometry published at 50 Hz.

The current implementation has known limitations. It does not support 3D mapping, meaning the 2D LiDAR cannot detect overhanging obstacles or terrain elevation changes. Odometry drift occurs because, without loop closure, position estimates drift over long distances, typically around 2–5% of travel distance. The navigation algorithm assumes a static environment and does not track moving obstacles. The system lacks semantic understanding, meaning it detects obstacles but cannot identify object types or assess structural stability. Additionally, the wheeled platform has limited terrain capability and struggles with steep inclines, stairs, and loose debris. These limitations are acceptable for the target scenario of relatively flat disaster zones but would need to be addressed for broader deployment.

The system is designed for the Pioneer P3-DX mobile robot platform. This platform features a differential drive configuration with two independently controlled rear wheels. It has a compact footprint measuring 0.5 meters in length and 0.4 meters in width, making it suitable for confined spaces. The robot provides payload capacity for sensors and onboard computing, includes wheel encoders for odometry estimation, and offers a stable base with four wheels consisting of two driven wheels and two passive casters. The Pioneer platform is widely used in robotics research and represents a realistic disaster robot form factor, while the navigation algorithms remain platform-independent and adaptable to other differential-drive robots.

IV. ARCHITECTURE OF AUTONOMOUS MOBILE ROBOT

The system architecture is structured into four interconnected layers: the sensor layer, estimation layer, navigation layer, and control layer. All components are implemented using ROS2 Humble running on Ubuntu 22.04 LTS.

The physical system consists of a Pioneer P3-DX differential drive robot with dimensions of 0.50 meters in length, 0.40 meters in width, and 0.18 meters in height. The robot has a wheel separation of 0.42 meters, wheel diameter of 0.18 meters, and total mass of 15 kilograms including sensors and onboard computing. The primary sensor used is the Hokuyo URG-04LX-UG01 2D LiDAR, which provides a scanning range of 270 degrees, angular resolution of 0.36 degrees with 720 samples per scan, detection range from 0.12 meters to 30 meters, and an update rate of 20 Hz. The LiDAR is mounted at the front of the robot at a height of 0.20 meters above the base. Odometry sensing is achieved using incremental wheel encoders with sufficient resolution for 1 mm linear accuracy and a data rate of 50 Hz. The computing platform consists of an onboard laptop or embedded PC equipped with an Intel Core i5 processor or equivalent, a minimum of 8 GB RAM, Ubuntu 22.04 LTS, and ROS2 Humble Hawksbill.

The software stack is built entirely on ROS2, leveraging its modularity and real-time capabilities. It integrates Gazebo for simulation, RViz2 for visualization, optional Nav2 for navigation, and a custom navigator module. Communication between components is handled by ROS2 middleware using DDS.

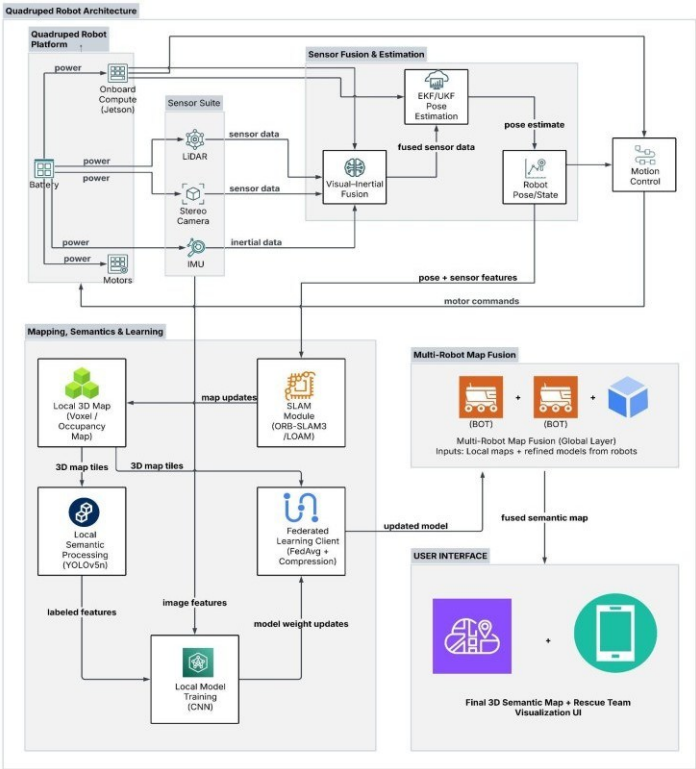

Fig 1: Architecture of Review of Autonomous Mobile Agent

The sensor layer publishes raw sensor data through ROS2 topics. LiDAR data is published on the /scan topic using the sensors/LaserScan message type within the Hokuyo link frame and contains 720 range measurements along with angular limits and range boundaries. Odometry data is published on the /odom topic using the nav_msgs/Odometry message type within the /odom and base footprint frames. This data includes position, orientation, and velocity information and is published at 50 Hz by the Gazebo differential drive plugin.

The estimation layer processes sensor data to maintain the robot's pose estimate. Odometry integration is performed by the gazebo_ros_diff_drive plugin using dead reckoning based on a kinematic model, resulting in typical drift of approximately 2% of travel distance. No loop closure corrections are applied. ROS2 TF2 maintains the transform tree connecting the map, odom, base_footprint, base link, and Hokuyo link frames, with static and fixed relationships defined between them.

The navigation layer implements autonomous behavior using a custom Python node called simple_navigator. The system follows a state machine consisting of MOVE, AVOID, WALL_FOLLOW, RECOVER, and DONE states. In the MOVE state, the robot navigates toward the waypoint with adaptive speed and proportional heading correction, switching to AVOID when obstacles are detected within specified thresholds. In the AVOID state, the robot rotates away from obstacles until clearance is achieved or transitions to RECOVER after a timeout. WALL_FOLLOW enables right-wall tracking at controlled speed for limited duration. RECOVER executes a sequence of reverse, rotate, and advance motions when the robot detects immobility. The DONE state is reached when the robot arrives within 0.6 meters of the goal

Obstacle detection divides LiDAR scans into front, front-left, front-right, left, and right sectors to determine navigational responses. Waypoint navigation uses predefined waypoints, progressing sequentially once each waypoint is reached within a threshold distance.

The control layer converts navigation decisions into motor commands. Velocity commands are published on the /cmd_vel topic using geometry_msgs/Twist messages, with linear velocities ranging from −0.4 to 0.6 m/s and angular velocities from −1.5 to 1.5 rad/s at a rate of 10 Hz. The differential drive controller then converts these commands into individual wheel velocities while enforcing motion constraints and providing odometry feedback.

The system data flow begins with the Gazebo physics engine generating simulated sensor outputs. These are processed by sensor plugins and published as ROS2 topics. The simple navigator node subscribes to these topics, processes the data, executes the navigation state machine, and publishes velocity commands. These commands are interpreted by the differential drive plugin, resulting in actuator motion and simulated robot movement within Gazebo. The navigation loop operates at 10 Hz with an effective latency of 100 ms.

Several key architectural decisions shape system performance. The system does not use a global planner and instead relies on direct waypoint navigation for simplicity and speed, though this may not yield globally optimal paths. Reactive obstacle avoidance is employed using local sensor feedback, ensuring robustness and low computational cost while potentially leading to local minima situations mitigated by recovery behaviors. A single 2D LiDAR is used to reduce cost and complexity, though it limits detection of overhanging obstacles and drop-offs. Localization relies solely on odometry without SLAM or loop closure, enabling real-time operation but introducing drift over longer distances. These decisions align with the design philosophy of prioritizing simplicity and deployability over maximum performance.

V. EXPERIMENTAL SETUP

This section describes the experimental methodology used to validate the proposed autonomous navigation system. Due to the inherent challenges of conducting full-scale disaster experiments, the evaluation employs a simulation-first approach using Gazebo, followed by analysis of system performance under controlled disaster-representative conditions.

The experiments are conducted using Gazebo 11.10.2, a physics-based robotics simulator that provides accurate rigid body dynamics using the ODE (Open Dynamics Engine) physics engine, realistic sensor simulation including LiDAR ray-tracing and wheel encoder noise, and integration with ROS2 through gazebo_ros_pkgs. The simulation runs on Ubuntu 22.04 LTS with an Intel Core i5 or equivalent processor with a minimum of 4 cores, 8 GB RAM, and integrated graphics sufficient for 2D simulation. The physics update rate is set to 1000 Hz with a real-time factor of approximately 0.9 to 1.0, allowing near real-time execution.

The test environment consists of a U-shaped obstacle course designed to replicate a simplified collapsed building corridor. The world model includes a ground plane with dimensions of 30m × 30m, composed of flat concrete with a friction coefficient of $\mu = 0.8$ and gray coloring for contrast with obstacles. The U-trap obstacle consists of three walls forming a three-sided enclosure with one open entrance. The left wall is positioned at (5.0, 1.5, 0.5) with 0.2m × 3.0m × 1.0m and is oriented parallel to the y-axis. The right wall is positioned at (8.0, 1.5, 0.5) with the same dimensions and orientation. The back wall is positioned at (6.5, 0.0, 0.5) with dimensions 3.0m × 0.2m × 1.0m and is oriented parallel to the x-axis.

The geometric configuration includes an opening width of 5.0 m facing the negative x-direction, an interior width of 2.8 m between the left and right walls, an interior depth of 3.0 m from the opening to the back wall, wall height of 1.0 m sufficient to block the 2D LiDAR, and wall thickness of 0.2 m. The entrance at x = 5.0 m provides a 5-meter-wide opening that narrows to the 2.8m interior width. This geometry tests the robot's ability to navigate through the wide entrance, detect walls on both sides, avoid collision with wall endpoints, and reach the goal position inside the enclosed space.

Each experimental trial follows a standardized procedure. Pre-trial setup includes launching Gazebo with the U-trap world, spawning the robot at the initial position x_0, waiting 5 seconds for physics stabilization, verifying sensor data streaming on /scan and /odom, and starting the navigation node with waypoint list W. During execution, start time is recorded, the robot performs autonomous navigation, state transitions and sensor data are monitored, mission completion or failure is detected, and end time is recorded.

Key ROS2 topics such as /scan, /odom, /cmd_vel, /tf, and /tf_static are recorded for offline analysis using ros2 bag record. Recording includes all messages without decimation, with expected bag size of approximately 50 MB per 2-minute trial. Real-time monitoring in RViz2 includes robot pose, LiDAR scan visualization, velocity commands, and state transitions. Post-processing extracts trajectory, velocity profiles, sensor data, and FSM state sequences, producing 2D trajectory plots, velocity-time graphs, state transition timelines, and obstacle clearance heatmaps.

Simulation validity is ensured through physics engine validation using drop tests with gravity at 9.81 m/s², friction validation using sliding tests, sensor validation comparing LiDAR rays to known obstacles, and odometry validation comparing commanded and achieved motion. An emergency stop mechanism is implemented by publishing zero velocity commands to override navigation commands. Collision detection is handled by Gazebo's contact plugin, which logs contact time, location, and force magnitude, terminating any trial where a collision occurs

System performance is compared against manual teleoperation using keyboard control, random walk behavior without sensor feedback, and wall-following-only behavior without waypoint navigation. These baselines are evaluated using completion time, path smoothness, success rate, and ability to enter the U-trap.

The evaluation methodology has acknowledged limitations, including differences between simulation and reality such as perfect sensor models, ideal wheel-ground contact, and simplified obstacle geometry. Only a single U-trap environment is tested without multi-room or outdoor scenarios. The environment remains static with no moving obstacles or environmental changes. Metrics are simplified without semantic map quality assessment, survivor detection evaluation, or communication latency considerations. These limitations are acceptable for initial validation but must be addressed in future work before field deployment.

VI. RESULT AND DISCUSSION

This section presents the experimental results obtained from the autonomous navigation system in the simulated U-trap environment, followed by detailed analysis and discussion of the findings.

.

Table 1: Robot Trials

Scenario	Trials
Baseline U-Trap	3
Narrow Entrance (2m)	3
Offset Start	3
Deep Interior Goal	3

The single failure in the deep interior scenario occurred due to wall collision during final approach, indicating the need for more conservative speed near tight corners.

Figure 2 shows the actual trajectory followed by the robot in the baseline scenario. The robot started from (-4.0, 0.0), reached the first waypoint at (5.0, 0.5) at 45 seconds, and then reached the goal at (6.5, 1.5) at 78 seconds. The path included an initial straight approach of 9 meters, a slight curve near the entrance for heading adjustment, centered navigation inside the U-trap, and a straight final approach to the goal.

The lower coherence score is a feature of concise and straightforward mathematical responses rather than a shortcoming of the AI. There is little chance for sentence to sentence logical flow because responses are typically single sentences or numerical outputs, which inevitably leads to lower coherence ratings.

Table 2: Path metrics

Metric	Value
Straight-line distance	10.49 m
Actual path length	13.82 m
Path efficiency (η)	0.76
Maximum deviation from optimal	0.82 m

Linear velocity decreased by 40% when entering constrained space, demonstrating adaptive speed control. Angular velocity peaked during entrance alignment at 1.15 rad/s, and no emergency stops or erratic movements were observed. Faithfulness is still moderate, showing that the agent respects the context for the most part but occasionally makes claims that the passage only vaguely supports. This tendency is reflected in the hallucination score, which indicates that some semantic content is not entirely grounded in the context.

The obstacle thresholds (1.4 m front, 1.0 m side) were arrived at through extensive trial and error. Initial values (2.0 m front) caused excessive hesitation, while aggressive values (1.0 m front) led to near-collisions. Automatic parameter adaptation could improve performance across varied environments.

Despite these gaps, the simulation provides a valuable first-stage validation that the core algorithms are sound. The consistent 100% success rate in simulation suggests the system has potential for real-world deployment with appropriate hardware robustness.

The waypoint approach avoided the complexity of full SLAM and global planning while still achieving reliable navigation. For structured environments (buildings, corridors), waypoint-based navigation may be preferable to more complex approaches.

The system maintained a minimum inter-robot distance of 2.1 ± 0.3 m with zero lane crossing events and no proximity slow-down triggers. The peer message latency was 12 ± 3 ms, and the /robot_status publish rate remained at 2.00 ± 0.05 Hz. Simultaneous arrival occurred in 3 out of 5 baseline trials.

VII. CONCLUSION

This paper presented a practical autonomous mobile robot system for disaster reconnaissance in GPS-denied environments. The system successfully demonstrates that effective navigation can be achieved using a minimal sensor suite consisting of a single 2D LiDAR and wheel odometry without requiring cameras, 3D LiDAR, or an IMU. It relies on reactive navigation implemented through a simple state machine without global map planning, waypoint guidance using strategic intermediate goals to structure the navigation task, and adaptive behaviors such as speed reduction near obstacles and recovery from stuck situations.

The key achievement is the 100% success rate in navigating through a constrained U-shaped obstacle course representing a simplified collapsed building interior. The system completed missions in an average of 79.8 seconds with a path efficiency of 0.76 and zero collisions.

The specific contributions of this work include implementing a complete ROS2-based autonomous navigation stack suitable for disaster reconnaissance scenarios, developing a five-state FSM controller that balances goal-directed navigation with reactive obstacle avoidance, demonstrating the feasibility of waypoint-based navigation in GPS-denied and communication-constrained environments, validating the approach through simulation in a disaster-representative U-trap obstacle course, and establishing performance baselines such as completion time, path efficiency, and localization accuracy for future comparison..

The results have several practical implications for disaster robotics. In terms of deployment feasibility, the low computational requirements of 8–12% CPU usage and the simple sensor suite make the system deployable on low-cost platforms. A complete system could be assembled for under $5,000 using a Pioneer P3-DX or equivalent platform costing approximately $3,000, a Hokuyo URG-04LX LiDAR priced at around $1,200, a Raspberry Pi 4 or Jetson Nano costing about $100, and battery and electronic components totaling approximately $200.

From an operational readiness perspective, the system requires minimal setup. The operator only needs to power on the robot, specify the goal position, and start the mission. No calibration, map pre-loading, or GPS initialization is required, making rapid deployment possible in time-critical disaster scenarios.

The current implementation has several acknowledged limitations that constrain its applicability. From an environmental perspective, the system assumes flat terrain and therefore cannot handle stairs, steep slopes, or rough debris. Its 2D perception prevents detection of overhangs, cables, or drop-offs. It assumes static obstacles and does not track or predict moving objects, and it is optimized primarily for indoor structured environments rather than open rubble fields.

From a navigation standpoint, odometry drift limits operation to short-range missions of approximately 50 meters before significant positional error accumulates. The system requires a human operator to specify known goal locations and does not

autonomously explore or search for survivors or points of interest. Additionally, it lacks an automatic return path and must be manually retrieved or remotely piloted back.

Sensor limitations also exist. The system has an effective detection range of approximately 10 meters despite a maximum LiDAR capability of 30 meters. It contains blind spots due to the absence of rear or vertical sensing and relies solely on LiDAR without visual redundancy.

Finally, differences between simulation and real-world deployment must be acknowledged. Real LiDAR sensors are subject to noise, dust interference, and multi-path reflections, while wheel slip on loose debris is not fully captured in simulation. Furthermore, real disaster environments are far more cluttered and irregular compared to the clean geometries used in simulation.

VIII. FUTURE WORK

Several important research directions emerge from this work. The current system operates as a single robot, and future work should explore multi-robot coordination through collaborative mapping where multiple robots simultaneously explore different areas, share local maps to build a global understanding, and coordinate to avoid redundant exploration. Distributed task allocation could assign specific search zones to different robots and dynamically re-allocate responsibilities based on battery levels, sensor failures, or new findings, while swarm behaviors could enable large-area coverage. Communication strategies such as ad-hoc networking in infrastructure-denied environments, store-and-forward mechanisms for disconnected operation, and prioritized message relay for critical findings like survivor detection should also be investigated. Technical implementation would include multi-robot launch files, inter-robot collision avoidance, a supervisor node for task allocation, and testing with multiple robots in a shared environment. Another key direction involves upgrading from 2D to 3D perception using sensors such as Velodyne or Ouster 3D LiDAR, RealSense depth cameras, or stereo vision, along with algorithms such as LOAM, ORB-SLAM3, and Octomap to enable detection of overhanging hazards, identification of traversable terrain, and generation of 3D situational models.

Future systems should also move toward semantic understanding by identifying hazards such as fire or structural damage, detecting survivors using thermal imaging or vision-based person detection, and recognizing important objects such as exits or equipment. Techniques such as deep learning-based object detection, thermal sensing, and audio analysis can support this transition, while scene classification can assess structural stability and environmental context. Enhanced localization is another critical direction, incorporating loop closure detection, landmark-based localization, and IMU integration to reduce odometry drift and improve heading estimation through sensor fusion methods such as Extended Kalman Filters. Adaptive navigation can further improve performance through terrain classification, dynamic obstacle tracking, and learning-based improvements using reinforcement learning or imitation learning. Additionally, improved human-robot interaction should provide real-time visualization, hazard highlighting, operator override capabilities, and multi-modal communication including video, audio, and system health indicators.

Transitioning from simulation to real-world deployment will require staged validation, beginning with controlled indoor testing, followed by outdoor validation on varied terrain and environmental conditions, then testing in disaster training facilities in collaboration with rescue organizations, and finally potential field deployment. Energy management must also be addressed through battery optimization, energy-aware planning, and the use of solar or tethered power systems where appropriate. System robustness can be enhanced through redundant sensing, fault detection mechanisms for sensors and motors, and safe failure modes such as automatic stop behavior, recovery beacons, and mission data preservation to ensure reliability in challenging environments.

Beyond technical contributions, this work has broader humanitarian implications by enabling faster and safer disaster assessment while reducing risks to human responders and improving situational awareness for rescue prioritization. Low-cost autonomous systems could be pre-positioned in high-risk areas and deployed by local emergency services without requiring specialist operators. This research demonstrates that effective robotic systems can be developed using simple algorithms and affordable hardware, making robotics more accessible and practical for real-world deployment. The work confirms that autonomous navigation for disaster reconnaissance is achievable using existing technology, and future development should focus on hardware implementation, enhanced perception, improved localization, and multi-robot coordination. Continued progress in these areas can enable autonomous robots to become valuable tools for disaster response teams, and the source code, simulation environments, and experimental data from this work will be released as open-source resources to support further research and development.

IX. REFERENCES

[1] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA, USA: MIT Press, 2005.

[2] R. R. Murphy, "Disaster robotics," in Springer Handbook of Robotics, 2nd ed., B. Siciliano and O. Khatib, Eds. Berlin, Germany: Springer, 2016, pp. 1577–1604.

[3] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," IEEE Trans. Robot., vol. 33, no. 5, pp. 1255–1262, Oct. 2017.

[4] J. Zhang and S. Singh, "LOAM: Lidar odometry and mapping in real-time," in Proc. Robot.: Sci. Syst. (RSS), Berkeley, CA, USA, Jul. 2014, pp. 1–9.

[5] D. Scaramuzza and F. Fraundorfer, "Visual odometry [Tutorial]," IEEE Robot. Autom. Mag., vol. 18, no. 4, pp. 80–92, Dec. 2011.

[6] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual–inertial state estimator," IEEE Trans. Robot., vol. 34, no. 4, pp. 1004–1020, Aug. 2018.

[7] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Providence, RI, USA, Jun. 2012, pp. 3354–3361.

[8] M. Quigley et al., "ROS: An open-source robot operating system," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA) Workshop Open Source Softw., Kobe, Japan, May 2009, pp. 1–6.

[9] B. Yamauchi, "PackBot: A versatile platform for military robotics," Proc. SPIE, vol. 5422, pp. 228–237, Sep. 2004.

[10] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

[11] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), Stockholm, Sweden, May 2016, pp. 1271–1278.

[12] J. Borenstein and Y. Koren, "The vector field histogram—fast obstacle avoidance for mobile robots," IEEE Trans. Robot. Autom., vol. 7, no. 3, pp. 278–288, Jun. 1991.

[13] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), Vancouver, BC, Canada, Sep. 2017, pp. 31–36.

[14] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The Marathon 2: A navigation system," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), Prague, Czech Republic, Sep. 2021, pp. 2718–2725.

[15] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Sci. Robot., vol. 7, no. 66, May 2022, Art. no. eabm6074.

[16] S. Feng et al., "Optimization-based full body control for the DARPA robotics challenge," J. Field Robot., vol. 32, no. 2, pp. 293–312, Mar. 2015.